

Contents

1. Introduction	1
2. Architectural Foundations of the Implementation	2
2.1. Limitations of the Early Prototype and Choice of Implementation Language	2
2.2. Choice of Rule Language	3
2.3. Parallelism and Concurrent Execution	3
2.4. Network Model and Event List Synchronization	5
3. Event Log Implementation	6
3.1. External References	6
3.2. Meta-records	7
3.3. Data Record Format	8
3.4. Routing and Data Locality	9
3.5. Cryptographic Authentication and Signature Aggregation	9
3.6. Ensuring Integrity and Resilience to Hardware Failures	10
3.6.1. State File	11
3.6.2. Write Protocol	11
3.6.3. Crash Recovery	12
4. Transformation Rules	12
4.1. Pipeline Execution Model	12
4.1.1. Transition from a Pull Model to a Pipeline Push Model	13
4.1.2. Channel Types and Rule Placement Across Cores	13
4.1.3. Secondary Channel Adapters	14
4.1.4. Pipeline Organization	14
4.2. Persistent Storage of Internal States	15
4.2.1. Storage Requirements	15
4.2.2. Storage Model Based on Copy-on-Write Semantics	15
5. Faden Language Implementation	16
5.1. Compiler Architecture	16
6. Conclusion	17
Bibliography	18

1. Introduction

As part of the previous research work, a functional-reactive DBMS architecture was proposed, built on the basis of event-sourcing and using transformation rules to materialize a single final state regardless of the order of event processing. An incremental model of transformation rules and the requirement

of cache insignificance were formulated. Based on this model, a prototype in Haskell was implemented, confirming the theoretical viability of the architecture. Based on the prototyping results, the task was set to develop a specialized language, Faden, for describing DBMS rules and to design a high-performance DBMS implementation.

This work focuses on:

1. Developing an event log specification that accounts for physical organization, routing, and ensuring integrity during hardware failures.
2. Designing a disk storage for the internal state of transformation rules.
3. Developing a pipeline model for the execution of reactive rules.
4. Implementing the Faden language: a type system and a compiler that ensure the serializability, totality, and verifiability of DBMS rules.

2. Architectural Foundations of the Implementation

This section is devoted to examining the limitations of the prototype and selecting the implementation paradigm and tools.

2.1. Limitations of the Early Prototype and Choice of Implementation Language

In the previous work, Haskell was used to create the prototype. While Haskell served as an adequate prototyping tool, the transition to a production implementation revealed the following limitations:

- **Garbage collector usage:** the language relies on a garbage collector for automatic memory management by default, leading to unpredictable memory consumption and latency. In a multi-stage data processing pipeline, where the update process of certain reactive cells depends on the update results of dependency cells, such delays can quickly accumulate, causing performance degradation of the entire system.
- **Limited low-level control:** Haskell does not provide full control over low-level implementation details, limiting potential performance and optimization opportunities.
- **Limited ecosystem:** the Haskell ecosystem has only a limited set of libraries for direct interaction with I/O subsystems (`io_uring`, `O_DIRECT`).

To solve these problems, the Rust programming language was chosen as an alternative, meeting the requirements of high performance and providing full control over low-level implementation details. An important feature of Rust

compared to other candidates (such as C or C++) is its powerful static verification tooling, which provides safety guarantees in memory management.

2.2. Choice of Rule Language

As described in the previous work, the prototype used Haskell simultaneously as the implementation language and the rule definition language. Its continued use as a rule language entails problems identified in the previous work: the impossibility of serializing closures, the lack of totality guarantees and formal verification tools, and the allowance of cyclic data structures.

This work considers the task of implementing the Faden language compiler for the role of the rule definition language; however, its full integration into the DBMS is a subject for future work. Until the integration task is resolved, we will use Rust both as the implementation language and as the rule definition language.

2.3. Parallelism and Concurrent Execution

Many modern high-load systems rely on parallelism and concurrent execution to process multiple requests in short periods of time.

- **Parallelism** relies on the multi-core nature of modern processors to provide true simultaneous processing of multiple execution threads. While the potential performance gain is proportional to the number of additional logical cores, in practice it can be significantly limited: it is not always possible to perfectly divide tasks across independent logical cores, necessitating the use of atomic operations and locks for communication and access control to shared memory. Such coordination delays can significantly reduce performance, to the point where single-threaded processing might be faster in the presence of heavy contention.
- **Concurrency** implies the ability of a single core to be in the process of handling several independent requests, rapidly switching between them. Concurrent execution allows utilizing idle time (usually occurring when waiting for data from other cores, disk requests, or network communication) to perform work on independent tasks. This increases the overall throughput of the system but introduces overhead for task switching and can slow down the processing of individual requests.

Let us consider the following approaches to implementing parallelism and concurrency:

- The **sequential single-threaded** approach is the easiest to implement and serves as a stable baseline, but it poorly utilizes device resources and yields low performance. All requests are executed sequentially; any requests to the disk or network completely block the single processing thread.
- The **concurrent single-threaded approach** allows a single thread to process multiple requests by switching between them while waiting for I/O operations. This approach increases the overall throughput of the system with minimal trade-offs.
- The **concurrent multi-threaded approach with shared memory** assumes the existence of unified global data structures accessed from multiple threads. Thus, each thread processes its own set of incoming data, reading and updating a single global state, thereby implementing the idea of “communication through data sharing”. Such an approach requires careful use of locks and atomic primitives to ensure safe access with minimal thread blocking. Furthermore, an update of a portion of the shared data by one core can lead to cache invalidation for all other cores, slowing down reads.
- The **concurrent multi-threaded approach with memory isolation** (“shared-nothing”) means that each thread operates only on the data it directly owns, avoiding locks on global data structures. In such systems, channels are often used for communication between independent threads, involving the transfer of data ownership to the receiving thread. This implements the idea of “sharing data through communication”. This approach minimizes locks and contention, and also reduces the risk of cache invalidation, but requires strict data isolation and complex coordination. Often, this approach is combined with the Thread-per-core model, where each thread is pinned to a specific core, which further reduces cache invalidations and context-switching overhead.

The concurrent multi-threaded approach with memory isolation has been chosen as the model for the DBMS under development. The multi-stage pipeline of transformation rules, where the final state is materialized from the initial list of events, naturally maps to this architecture: each rule reads the results of the previous stage, updates its internal state, and passes the result to the next stage. In addition, exclusive ownership of the internal state and communication channels by a single thread allows for a strict FIFO data transmission model, guaranteeing strictly incremental updates of the DBMS internal state without the need to revert reactive cells to a previous state to handle an unordered stream of requests. However, memory isolation creates difficulties when transferring

intermediate results between stages: the architecture of reactive cells discussed in the previous work involves passing a reference to the full result of a given stage to subsequent pipeline stages, followed by calculating the list of changes at the receiving cell level. Such an architecture violates strict memory isolation. Resolving this contradiction requires a mechanism for computing changes on the source core rather than the receiving core, which will be discussed in section Section 4.1.

2.4. Network Model and Event List Synchronization

A distributed computer system is a collection of independent compute nodes that communicate with each other over a network and function as a single hardware and software resource from the user's perspective. Distributed systems can be used to solve various tasks:

- Service scaling, increasing the overall throughput of the system through horizontal scaling.
- Geographic distribution aimed at reducing telecommunication latency and improving service quality for the end user.
- Fault tolerance and reliability, the ability of the complex to continue functioning during a partial software or hardware failure.
- Ensuring security in decentralized “zero-trust” environments, where an increase in the total number of independent nodes enhances the network's resilience to targeted attacks by third parties.

As discussed in the previous work, the DBMS architecture under development relies on an event list as the source of truth, retroactively updating as new events are discovered. This feature allows the system to reach eventual consistency in the context of distributed computer systems, which only requires an external event list synchronization mechanism. The specific implementation of the synchronization mechanism can vary depending on the system requirements.

The previous work assumed the existence of a strict event order, based on which the StateGraph model was demonstrated. This approach remains a promising direction for further research; however, the current work aims to create a data storage and organization system that is invariant to the sequence of events.

Dropping the requirement for strict event ordering excludes both strict consensus models (such as Raft, Blockchain) and partial order models (such as Lamport clocks) from consideration.

Let us consider the issue of network reliability and trust:

- In a **trusted cluster**, all network nodes are controlled by a single operator and are therefore considered reliable sources of information. In such a system, verification (e.g., via cryptography) of synchronized events is not required, significantly simplifying the synchronization protocol and the on-disk data storage format (eliminating, for instance, the need to store a digital signature).
- In an **open network**, where nodes are controlled by independent operators, every node is potentially unreliable. Such a model is implemented, in particular, in the Nostr protocol: each event is signed by its author, and network members connect to a multitude of independent nodes to ensure network resilience against censorship through redundancy.

A trusted cluster model was selected for the current stage of work. The system under development pursues the goal of fault tolerance while maintaining architectural simplicity, but it does not aim to establish trust in a zero-trust environment. In the future, it is possible to replace the specific transport or add several independent ones without altering the internal data processing model.

3. Event Log Implementation

Since the event log is the sole source of truth in the DBMS under development, its integrity and durability are priorities. To ensure the log's preservation, append-only semantics are applied, where all entries are written strictly to the end of the log without modifying or deleting old events.

For optimal performance within the “shared-nothing” architecture, the unified event log is physically partitioned into several complementary logs. Each device core is bound to its own physical log and owns its set of events. Each physical log is segmented into sequential files, where each file stores sequential records, simplifying compaction and archiving.

Auxiliary indices are updated in the background based on the log entries.

3.1. External References

In an event-sourced system, the natural application of events is to describe direct actions or operations performed by the user. These operations can often reference previously defined objects. For example, in a system designed for storing editable documents, it is natural to represent an “edit” operation as a separate event that references a previously defined “document” object.

In order for the system to efficiently track interactions with these objects, it is proposed to implement a referencing system explicitly, storing external references used by the event separately from the rest of the body.

It should be noted that it is not reliably known which objects will be reused and which are mentioned only in a single event. Furthermore, storing “objects” and “events” separately increases system complexity and potentially requires the creation and maintenance of additional indices. To avoid overhead, a model of objects embedded within events with the possibility of external references is proposed.

In this model, every event that defines a new object embeds it entirely within its payload at a specific offset following the main data block. If some new event B needs to reference an object defined in the body of an earlier event A, event B must store a full reference to event A along with the actual offset of the object within the body of A.

3.2. Meta-records

While network communication must rely on long globally unique identifiers to describe events and users, the storage and local processing of a large number of events can be optimized through the use of local, short 8-byte numeric identifiers, which are unique within the context of a specific physical log.

To store the associated identifiers in the physical log, a set of special records is used. These records begin with an 8-byte signature describing the type of the registered object:

- **SENDER_ID**: Contains a BLS12-381 public key ([u8; 48] — a 48-byte sequence), to which a monotonic `LocalSenderId` (u64 — a 64-bit integer) is mapped. The choice of the cryptographic scheme is justified in section Section 3.5.
- **MSG_TYPE_ID**: Contains a serialized event schema, describing:
 1. The schema of grouping fields. Grouping fields are an extension of the main event body and are required to ensure data locality within the Thread-per-core model, which will be further discussed in section Section 3.4.
 2. The schema of the main event body.
 3. A textual description of the event’s purpose.

Based on this schema, a hash is computed, to which a monotonic `LocalMsgTypeId` (u64) is mapped. This identifier is used to interpret the event body during deserialization.

- **GROUP_ID**: Contains a serialized combination of `LocalMsgTypeId` and specific values of the grouping fields. A hash is computed from the combination, to which a monotonic `LocalGroupId` (u64) is mapped. Thus, each group captures exactly one event type, binding it to a specific set of grouping fields.

In the context of the Thread-per-core architecture, such identifiers can be generated through a simple counter increment, providing monotonicity without gaps. The specific counter values at the moment of recording any object can then be trivially reconstructed from the physical log, since writing is also performed monotonically.

3.3. Data Record Format

Each event is saved to disk in a format consisting of a fixed-size header, followed by an array of external references and the event body.

The record header (`MsgHeader`) contains the following fields:

- `group`: `LocalGroupId` — the group affiliation identifier of the record. This parameter determines the specific event type and the set of grouping fields.
- `sender`: `LocalSenderId` — the identifier of the sender, i.e., the user who initiated the event.
- `external_refs_len`: u32 — the number of external references used in the event.
- `body_len`: u32 — the length of the event body in bytes.
- `source_id`: u32 and `source_core_id`: u32 — the identifier of the event-registering node within the context of a trusted cluster and the core that performed the event registration. Used for efficient synchronization of event lists in cluster environments.

Placed after the header are:

- An array of external references `external_refs`. References are represented using a typed enum, allowing the encoding of two types of objects:
 1. `Sender`: allows encoding references to existing senders in the `LocalSenderId` format.
 2. `Data`: allows encoding references to objects defined in earlier events in the `{ source_event: LocalEventId, data_offset: u32 }` format.
- The event body `body`, containing data in a serialized format.

Within the event body, objects are addressed via a 32-bit packed pointer, the most significant bit of which determines the interpretation:

- 0 defines embedded objects: the pointer indicates the offset of the object within the parent event.
- 1 defines external objects: the pointer describes the index in the `external_refs` external reference array.

3.4. Routing and Data Locality

In the context of a shared-nothing architecture, the division of a unified event log into independent physical parts is driven by:

1. Increasing write throughput. Distributing incoming events across independent cores increases the overall system throughput via parallelism.
2. Ensuring co-location of the internal memory of transformation rules and events. Since transformation rules, discussed in section Section 4, are distributed among the cores, deterministic event routing has the potential to reduce the need for inter-core communication, ensuring the local placement of all data necessary for request processing.

To implement a routing mechanism that ensures locality, the internal event data is divided into a “body” and “grouping fields”. All events with the identical set of “grouping fields” are assigned a single group and are routed to the same core. The target core is determined from the global hash of the group.

To further minimize inter-core communication, all objects referenced by events recorded on a core are also copied to that core.

3.5. Cryptographic Authentication and Signature Aggregation

For authenticating event authorship, the BLS12-381 digital signature scheme based on elliptic curves was chosen, using elements of the G_1 group (48 bytes) to describe the public key and elements of the G_2 group (96 bytes) to describe digital signatures.

The key advantage of BLS12-381 compared to other algorithms, such as Ed25519, is its support for aggregated signatures: multiple signatures of independent events by different authors can be aggregated into one that describes the entire set. This approach allows proving the legitimacy of the event set stored by the system with minimal memory overhead, keeping only a single 96-byte aggregated signature for the entire log segment. However, unlike the model of individual signatures for each event, this approach does not allow third parties to verify the legitimacy of individual events in the set without verifying the entire set as a whole.

These cryptographic guarantees can be used to ensure the integrity and availability of public data by:

1. Forming deferred event queues. In the event of a failure of the central trusted cluster, the function of receiving incoming events from users can be delegated to trusted third-party servers, which can queue incoming events while aggregating their digital signatures. Upon restoring normal operations, the accumulated data can be transmitted to the target cluster and cryptographically verified.
2. Replication. Third-party servers can replicate the public data of the main cluster, saving its events along with the aggregated signature. In the event of a complete system failure, these replicas can be used for partial data recovery.

Another advantage of aggregated signatures is decentralization, where the validity of the log is confirmed by all network members, and the compromise of a server's secret key does not lead to a complete loss of trust in the replicated log.

3.6. Ensuring Integrity and Resilience to Hardware Failures

While the append-only semantics of the log preclude the overwriting of previously written data, the mechanism for safely flushing to disk requires separate consideration to ensure the durability of committed records regardless of hardware failures. In the classical write-ahead logging (WAL) model [1], this problem is solved through preemptive writing to a separate log.

Modern file systems and storage drives do not provide atomic write guarantees for regions of arbitrary alignment and size, and in the event of a failure, partially written data can corrupt old information located in the same block. According to the NVMe specification [2], write atomicity during power loss is determined by the AWUPF (Atomic Write Unit Power Fail) parameter, which is 4 kilobytes or more on most NVMe drives. Similarly, the ATA specification [3] defines the physical sector size (Advanced Format) as 4 kilobytes, within which a write is atomic. In combination with direct I/O (the `O_DIRECT` flag), which eliminates intermediate buffering by the kernel, 4-kilobyte boundary alignment minimizes the probability of an incomplete write at the device level.

However, the listed properties of storage drives do not constitute strict software guarantees. A number of drives possess internal volatile buffers, the contents of which can be lost upon power failure. Thus, 4-kilobyte boundary alignment is a practical measure that reduces the likelihood of corruption, but does not

eliminate it entirely. The ultimate guarantee of integrity can be provided by implementing a double superblock.

We will rely on the assumption of page isolation, assuming that rewriting specific 4-kilobyte file pages does not affect other pages. This assumption is fundamental to many systems relying on disk storage, including PostgreSQL [4] and SQLite [5], and is supported by drive communication protocol specifications [2]: a write command addresses a specific set of logical block addresses (LBAs), and, assuming the drive firmware functions correctly, does not affect other regions.

3.6.1. State File

Located next to the log is a fixed-size state file (16 kilobytes), consisting of two alternating 8-kilobyte superblocks. Each superblock contains the following data:

- `epoch`: `u64` — a monotonically increasing epoch number, allowing comparison of the novelty of two superblocks.
- `complete_segments`: `u64` — the number of fully populated log segments.
- `complete_segment_pages`: `u64` — the number of successfully written 4-kilobyte pages in the last segment.
- `tail_data`: `[u8; 4095]` — an incomplete data page that has not yet formed a full page and is not ready to be moved to the last segment.
- `LocalMsgId` event counters for the given core and counters for the cores of other cluster members synchronized with the current one.
- Accumulated signatures `[u8; 96]` for the given core and for synchronized cores of other cluster members.
- `indexed`: `u64` — a counter describing the last `LocalMsgId` for which all indices have been saved to disk.
- CRC32 checksum for verifying the correctness of the superblock.

3.6.2. Write Protocol

We will place only whole 4-kilobyte pages into the main segment file. Incomplete pages are stored exclusively in the state file.

The procedure for adding new data:

1. New records accumulate in RAM, replenishing the current `tail_data` buffer.
2. When the accumulated buffer volume reaches 4 kilobytes, the formed full page is written to the end of the main segment file, and `tail_data` is emptied and ready to receive new data.

3. After processing a complete incoming batch of events, an `fdatasync` operation is performed on the written log segments, guaranteeing the complete writing of the formed segments to disk.
4. A new superblock is written into the state file in place of the inactive superblock, and an `fdatasync` operation is performed. The written superblock is designated as active.
5. Secondary indices are asynchronously updated from the written data.

The order of operations is essential to ensure a correct recovery procedure.

3.6.3. Crash Recovery

When recovering from a hardware failure, the following procedure is executed:

1. Both superblocks are read from the state file.
2. Each superblock is verified using the CRC32 checksum. If a superblock with an incorrect checksum exists, it is considered corrupted and discarded. Since writing a new superblock can only occur after the old one has been completely saved, a situation where both superblocks are corrupted is impossible in the context of our page isolation assumption.
3. From the valid superblocks, the superblock with the highest epoch number is selected and assumed as active.

4. Transformation Rules

In the previous work, the abstraction of reactive cells, Gear, was implemented. Reactive cells, or transformation rules, conceptually function as pure functions of the event list and the results of other cells, but in practice are implemented as state machines with internal state for efficient incremental processing of incoming data. The previous work also defined the requirement of cache insignificance, which guarantees the semantic correctness of the implemented rule.

This section is devoted to the pipeline model of executing transformation rules and the organization of the storage for their internal states.

4.1. Pipeline Execution Model

Since each Gear is inextricably linked to its internal state, within a shared-nothing architecture, it is natural to pin each Gear to a specific core. This section explores the pipeline model as a method of organizing communication between transformation rules in a memory-isolated system.

4.1.1. Transition from a Pull Model to a Pipeline Push Model

In the Haskell prototype, interaction between rules was implemented using a pull model: the computation of a certain Gear's dependencies was initiated by an explicit request from the Gear itself, which suspended its operation until the dependencies were computed. Such a solution is a primitive implementation of a dependency system and allows for passing additional arguments within the request, but in the context of a shared-nothing system, it entails the problem of elevated overhead and high latency.

According to the principles of the shared-nothing model, both sending a request and receiving a result are modeled as a message over a data transmission channel. While high throughput is characteristic of data transmission channels between cores, the destination core itself might be overloaded, and significant time intervals can pass between the moment the core receives the message via the channel and the start of its processing.

The push model, where data is routed from source to receiver without an explicit request, is technically less flexible but better suits the compositional nature of the DBMS and reduces the processing latency of the entire task. Thus, a pipeline model is proposed, in which each request to the DBMS is processed through the sequential handling of relevant data by a pre-materialized pipeline.

4.1.2. Channel Types and Rule Placement Across Cores

Each Gear takes input information via one or more communication channels linking it to its dependencies. Let us call one of these channels "primary" and the rest "secondary". Which channel is considered primary is left to the developer's discretion, but in general, it is reasonable to designate the channel through which the main array of data for processing is delivered as primary.

In this case, it is natural to introduce the following rule for distributing Gears across cores: a Gear is placed on the core where its primary input is located. Such a model makes it possible to significantly reduce the cost of inter-core communication when using trivial linear pipelines, and also allows the Gear to reap all the benefits of the core physically owning the data received via the primary channel.

At the same time, the sources of secondary communication channels can be located on other cores, requiring inter-core communication for data transmission. A significant difficulty is that full event tables or intermediate

Gear results are disk-based data structures, and their complete transmission over a communication channel is difficult:

1. Full materialization of the transmitted structure in RAM entails an enormous disk load and can quickly exhaust the server's RAM.
2. Sharing the source core's local block cache with the receiving core violates shared-nothing principles, complicating the application architecture and potentially leading to overall performance degradation.
3. The use of the receiving core's block cache to access the source core's storage can lead to duplicated I/O operations and increased RAM consumption.

To effectively solve this problem, the concept of adapters is proposed.

4.1.3. Secondary Channel Adapters

Note that in the incremental computation model, the receiving core of a secondary channel often does not need the complete dataset, but rather a pre-known subset of data or the data changes compared to the previous version. In this case, the source core can pre-process its result according to an established procedure (an adapter) and send exclusively relevant data over the communication channel.

An adapter can be implemented in two ways depending on the needs:

1. **Direct model:** the adapter is represented as a function that takes the new data of the source core as input, processes it within the context of its isolated memory, and forms a final self-sufficient representation that can be sent over the communication channel.
2. **Delta model:** similar to the direct model, but the adapter is represented as a function of two variables, also taking as input the source core's old data used during the last communication.

The delta model requires the source core to track separately what data it transmitted to the receiving core during the last communication, but it naturally maps to the incremental computation model.

4.1.4. Pipeline Organization

Upon receiving a request from an external system to read the result of a specific Gear, the following procedure is performed:

1. A unique task identifier, `TaskId`, is formed.
2. The dependency graph is recursively determined: a set of rules and event tables whose results are required to compute the requested rule.

3. Instructions are sent to all cores containing the pipeline input data (event tables), detailing the requested event groups, the receivers of this data, and further instructions to pass along. The data is processed by adapters if necessary and routed further down the pipeline.
4. Subsequent stages of the pipeline are Gears. They receive instructions, wait for all data to arrive, and begin execution, passing their results along with further instructions.
5. After several iterations, the data reaches the end of the pipeline, returning to the originating core.

4.2. Persistent Storage of Internal States

While the persistence of the Gear internal state storage is not a strict requirement, the ability to fully save the internal state allows the system to process massive volumes of data, vastly exceeding the size of RAM, and to quickly restore its state upon failures or restarts.

4.2.1. Storage Requirements

The storage of internal rule states must satisfy the following requirements:

- **Optimization for RAM operations:** reading and updating state in RAM is proposed to be considered the primary operating mode of the DBMS; disk operations must be performed in the background.
- **Copy-on-Write semantics:** the functional architecture of the DBMS heavily relies on the use of immutable data structures, for example, to detect changes in the context of the adapter delta model (described in section Section 4.1.3), and old data versions must remain unchanged on disk until they become unreachable.
- **Zero-copy:** on-disk data structures should be designed for direct mapping to native Rust structures without intermediate deserialization for rapid access.
- **Support for a wide range of types:** in the DBMS under development, the data storage format is not fixed, and rule developers can use various data structures to store internal state.

4.2.2. Storage Model Based on Copy-on-Write Semantics

The storage is organized as a heap — a space of logical pages with a fixed size of 4 kilobytes. Each logical page is identified by a unique PageId. Reference counting and Copy-on-Write models are used: every page remains immutable from the moment of allocation until the moment of garbage collection, which is triggered by the absence of active references.

At the top level sits a B+Tree, which maps each active Gear on a core to its storage. A Gear's storage consists of a reference to its latest state, as well as references to its previous states passed to the Gears depending on it.

Storage metadata is saved in a pair of superblocks using a principle analogous to that described in section Section 3.6.1. Each superblock contains:

- **Checksum hash** for correctness verification.
- **Generation** — a monotonic number for comparing the novelty of superblocks.
- **Reference to the root object.**
- **Reference to the memory map**, describing occupied and free disk space regions.
- **Reference to the recounting map**, listing pages that have more than one reference pointing to them.

5. Faden Language Implementation

In the previous work, requirements for a specialized DBMS rule language were formulated: serializability, totality, verifiability, and the prohibition of cyclic data structures. A draft of the Faden language was also presented, featuring extrinsic typing, dependent records, and support for formal verification tooling. This section focuses on the results of the Faden implementation.

5.1. Compiler Architecture

The Faden compiler is implemented in Haskell (~3000 lines) and consists of three sequential stages: normalization and type checking, erasure, and bytecode generation.

Type checking is performed in bidirectional mode [6]: for each expression, the compiler either synthesizes a type or checks the expression against an expected type (checking mode). The compiler automatically introduces and resolves existential variables, inferring types for complex expressions; it implements subtyping; and it supports rewrite rules.

Upon completion of type checking, all type information is erased and bytecode generation occurs, translating the program into instructions for a stack-based virtual machine. The instruction set includes: constant loading, deep variable copying, function application, closure creation with an explicit list of captured values, conditional branching, and record and list construction. The compilation

result is serializable into a binary format with support for full reverse transformation.

Table 1 presents all five previously discussed problems of using Haskell as a rule language, along with the specific Faden implementation mechanisms aimed at resolving these problems.

Problem	Fadeno Mechanism	Essence of Resolution
Closure serialization	Full language serializability	All language expressions are fully serializable, including closures, represented as combinations of captured variables and an instruction set
Lack of totality guarantees	loop with decreasing measure proof	The only recursion mechanism requires a type proof $\text{Where } (\text{measure next} < \text{measure curr})$; non-negativity of the measure guarantees termination
Lack of formal verification	Propositional equality	Ability to prove invariants and equivalence directly in the code; verified by the compiler
Cyclic data structures	Runtime environment restriction	The runtime environment does not provide tools that allow implementing cyclic data structures
Complexity	Extrinsic typing	Verification constructs (proofs, annotations) exist exclusively at the checking stage and, strictly speaking, are not required to run the program

Table 1: Correspondence between rule language problems and Faden implementation mechanisms

The full integration of Faden into the DBMS is a subject for future work.

6. Conclusion

In this work, the task of designing a high-performance functional-reactive DBMS architecture based on event-sourcing was solved.

1. An event log specification was designed, including: a physical segmentation model across processor cores, on-disk record format, support for external

references and embedded objects, mechanisms for cryptographic authentication and signature aggregation, as well as mechanisms to ensure integrity during hardware failures.

2. A persistent storage for the internal states of transformation rules based on Copy-on-Write semantics was designed, optimized for RAM operations and satisfying zero-copy mapping requirements.
3. A pipeline push model for executing reactive rules in a shared-nothing architecture was developed, including a rule for placing Gears on cores based on the primary channel, as well as a secondary channel adapter mechanism for efficient incremental transmission of changes between cores without violating memory isolation.
4. A Faden language compiler (~3000 lines of Haskell) was implemented, ensuring the serializability, totality, and verifiability of DBMS rules through full language serializability, a provable recursion mechanism, propositional equality, and extrinsic typing.

Future directions of work:

- Finalization and testing of the described architecture.
- Full integration of the Faden language into the DBMS.

Bibliography

- [1] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1992, p. 1070.
- [2] NVM Express, Inc., “NVM Express Base Specification.” 2020. [Online]. Available: <https://nvmexpress.org/specifications/>
- [3] INCITS Technical Committee T13, “Information technology – ATA/ATAPI Command Set - 4 (ACS-4),” Washington, DC, USA, Standard INCITS529–2018, 2018.
- [4] PostgreSQL Global Development Group, “PostgreSQL Documentation, Chapter 20: Reliability and the Write-Ahead Log.” [Online]. Available: <https://www.postgresql.org/docs/18/wal.html>
- [5] SQLite Contributors, “SQLite File Format: Write-Ahead Logging.” [Online]. Available: <https://www.sqlite.org/wal.html>

- [6] J. Dunfield and N. R. Krishnaswami, “Complete and Easy Bidirectional Typechecking for Higher-Rank Polymorphism.” [Online]. Available: <https://arxiv.org/abs/1306.6032>